



Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

Computer Science II

Inheritance in C++

Dr. Paul Bustamante y Ph.D. Jesús Paredes

- 1. Review**
- 2. OOP characteristics**
- 3. Inheritance**
- 4. Derived class constructors**
- 5. Conversions between objects of base classes and derived classes**
- 6. Multiple inheritance: virtual base classes**

2. OOP characteristics

Fundamental characteristics of Object Oriented Programming (**OOP**) include :

- **Abstraction:** The representation of the essential characteristics of something without including background or irrelevant details.

The **class** is an abstraction because it defines the generic properties and attributes of a set of objects.

- **Encapsulation or concealment of information:** Variables and member functions of a class can be declared as **public**, **private** or **protected**. In this way, access to class members can be controlled and inappropriate use can be avoided.

- **Inheritance:** It is the mechanism for automatically **sharing methods and attributes** between classes and subclasses. A class can be derived from another class, and in this case inherits all variables and member functions.

- **Polymorphism:** This feature allows multiple forms of the same method to be implemented, each depending on the class on which the implementation is performed.

3. Examples of Attribute Inheritance



Person

- **Passport**
- **Name**

Student

- **Passport**
- **Name**
- Course
- Mark

Teacher

- **Passport**
- **Name**
- Type: profesor, tenured, assistant

Erasmus

- **Passport**
- **Name**
- Course
- Mark
- Country

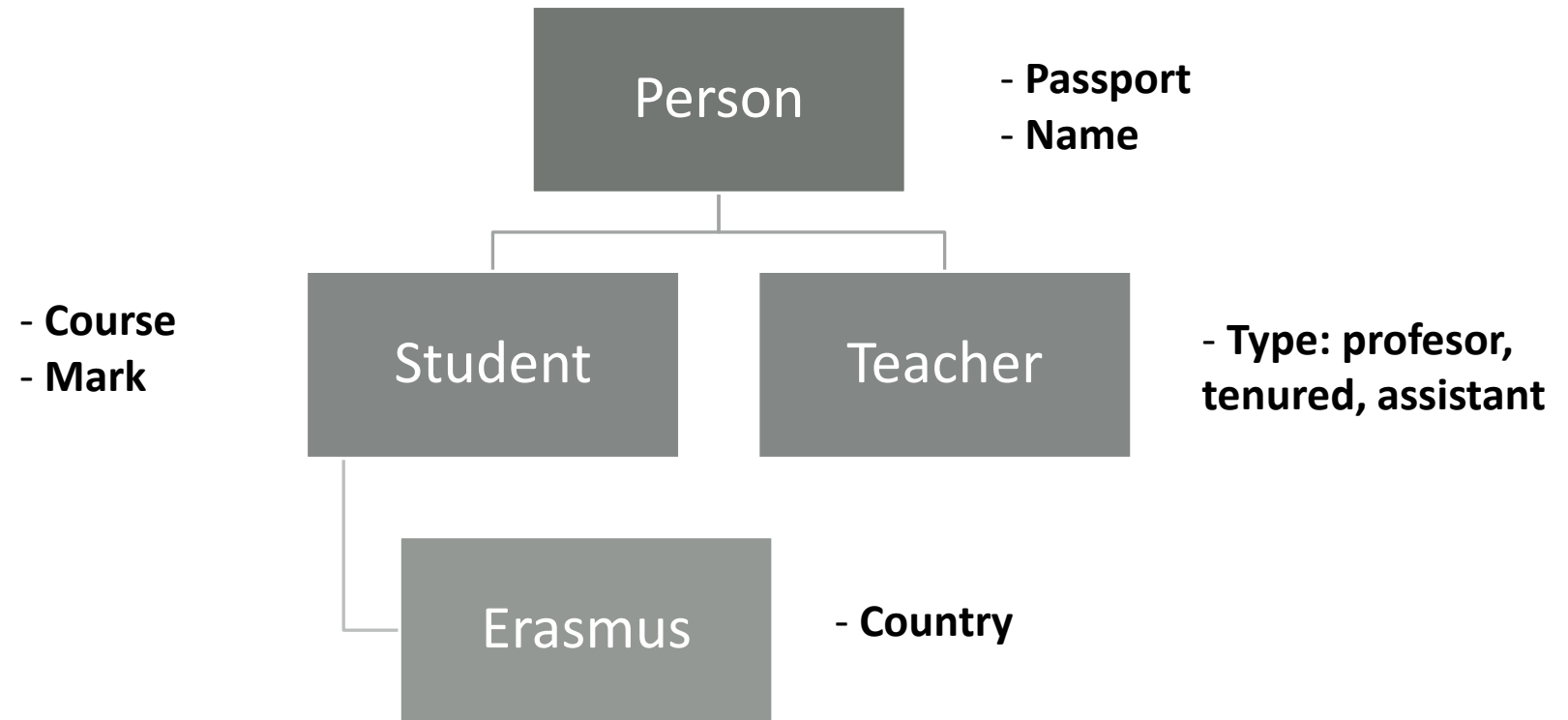
3. Examples of Attribute Inheritance



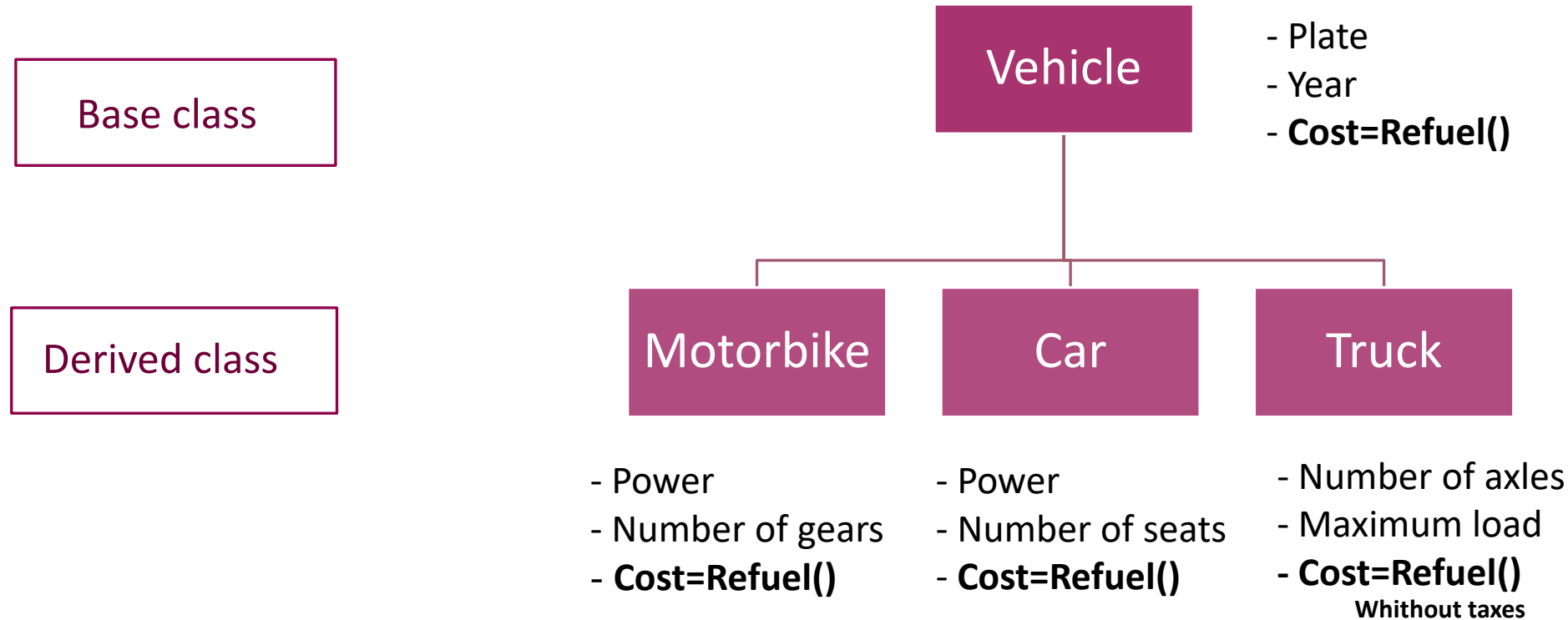
Base class

Derived class

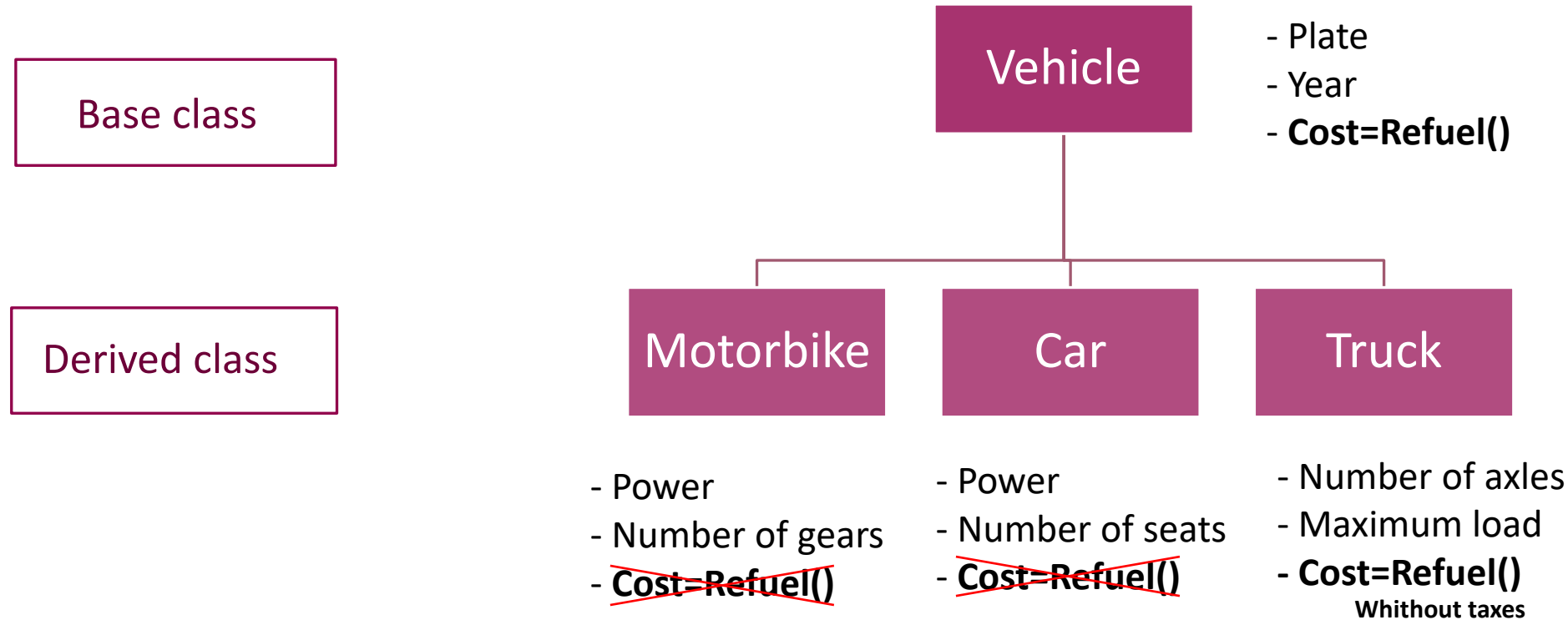
Derived class



3. Examples of Method Inheritance



3. Examples of Method Inheritance



3. Characteristics of inheritance

Main characteristics

- It can be defined from another existing class (base class), from which it inherits its variables and its member functions.
- You can **add** and/or **redefine** new variables and/or member functions.
- The **base class** is usually more general than the **derived class**. The derived adds new determinations or specifications (new variables and/or member functions).
- At the same time, the **derived class** can be the base class of a new **derived class**, which inherits its variables and member functions. A class **hierarchy** can be formed.

Types of variables and functions

- **Protected** members, like **private** members, are not accessible from outside the class.
- In a **base** class, protected members differ from **private** members in the way that they can be accessible to **derived classes** from that **base class**.

	public	protected	private
Main	✓	✗	✗
Derived class	✓	✓	✗

3. Characteristics of a derived class

How to define inheritance

```
class Base{  
    // Base class variables and functions  
};  
  
class Derived : public Base{  
    // Derived class variables and functions  
};
```

Where they must be defined. Two options:

- All classes (base and derived) in the same “.h” → “Base.h”
- One “.h” for each class → “Base.h”, “Derived.h”,

Types of inheritance: For the *derived class*, the *base class* can be inherited as **public** or **private**:

- If the *base class* is inherited as **public**, the *derived class* inherits the **public** and **protected** members of the *base class* as **public** and **protected** members, respectively.
- If the *base class* is inherited as **private**, the *derived class* inherits all members of the *base class* as **private**.

3. Example: class "employee"

Imagine that a company has three types of employees:

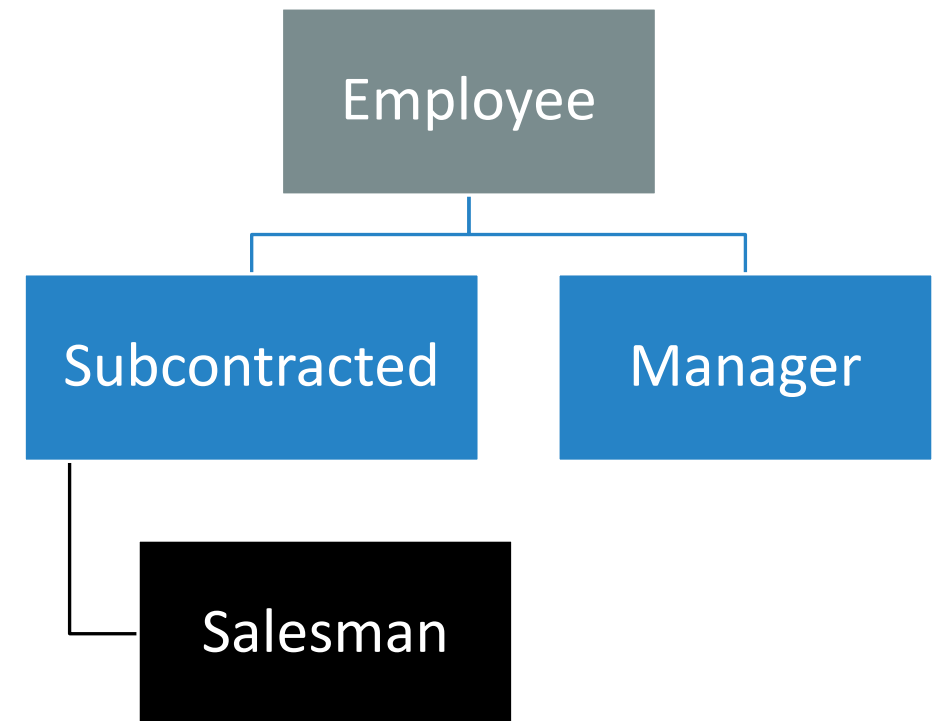
- **subcontracted**: charges per hour.
- **salesman** : charges per hour, plus a commission on sales.
- **manager**: charges a fixed amount per month.

You want to make a function called **calculate_payment ()** that calculates how much to pay each employee each month.

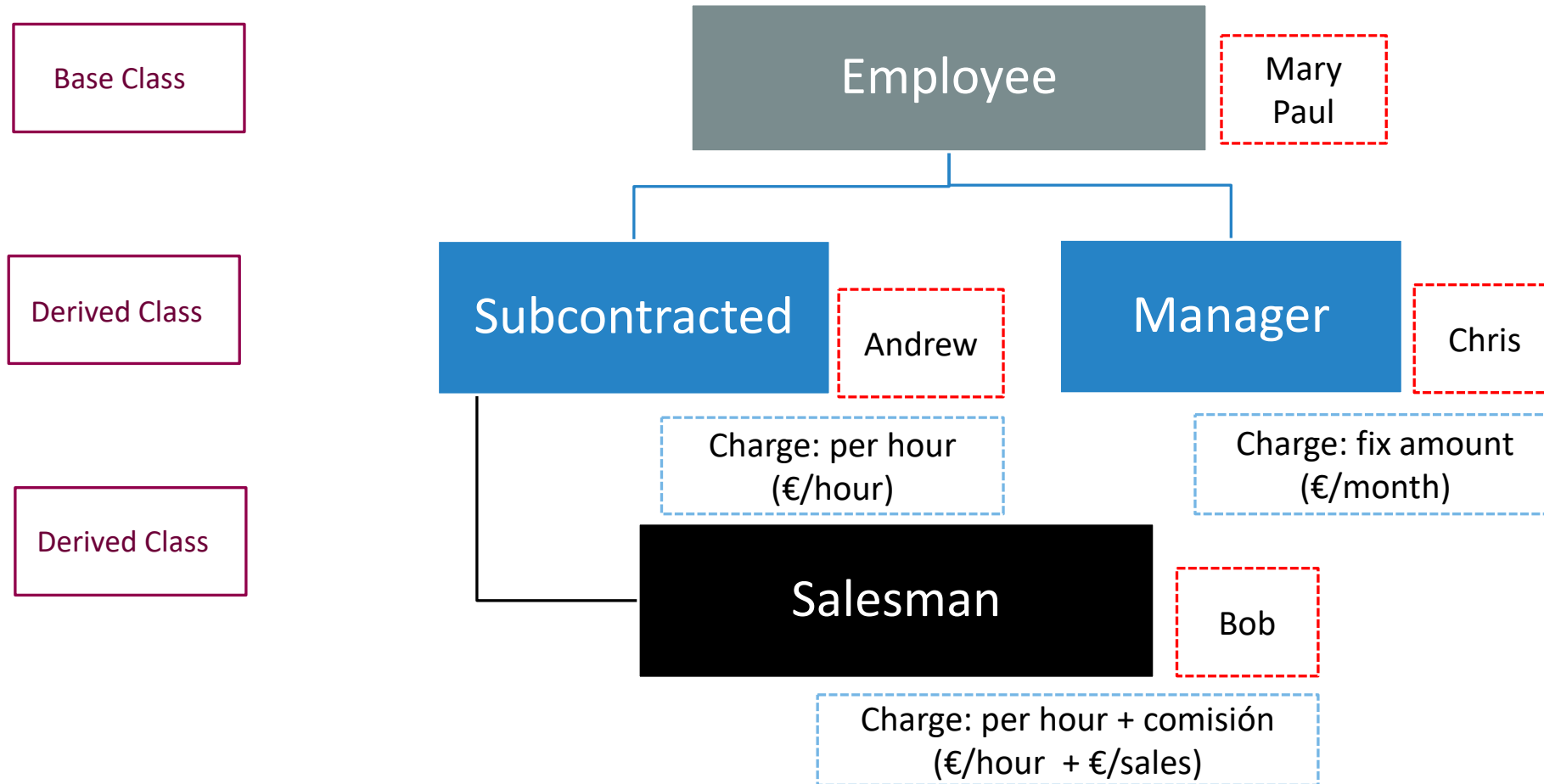
You can think of a **hierarchy** of *employees* as shown in the figure.

The **principles of this hierarchy** are:

- All are **employees**.
- **Salesmen** are paid per hour like the **subcontracted** ones(in fact they are a special type of **subcontracted**), but they also have a commission, so they can be considered as a particularization of **subcontracted**.
- The only thing the **managers** have in common with the others is the fact that they are **employees**.



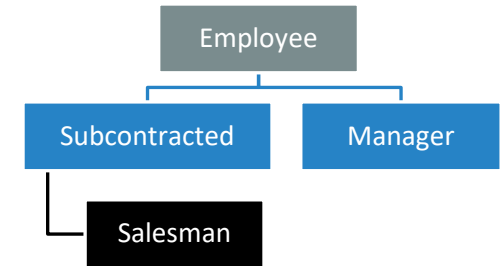
3. Example: class "employee"



3. Example: class "employee"

```
class employee
{
private:
    char *name;
public:
    employee();
    employee(char* n);
    char* return_name();
    float calculate_payment() { return 0.0; }
};
```

```
class manager : public employee
{
private:
    float salary;
public:
    manager();
    manager(char* n, float s);
    void Set_salary(float s);
    float calculate_payment();
};
```



```
class subcontracted : public employee
{
private:
    float cost_rate;
    float hours;
public:
    subcontracted();
    subcontracted(char* n, float c, float h);
    void Set_cost_rate(float c);
    void Set_hours(float h);
    float calculate_payment();
};
```

```
class salesman : public subcontracted
{
private:
    float commision;
    float sales;
public:
    salesman();
    salesman(char* n, float cr, float h, float c, float s);
    void Set_commission(float c);
    void Set_sales(float s);
    float calculate_payment();
};
```

3. Example: class "employee"

Remember that each **derived class** inherits all the members of the corresponding **base class**.

The **salesman** class inherits the members of the classes **subcontracted** and **employee**.

The **derived class** can access to **protected** members of the base class, but can NOT directly access the **private** member variables of the **base class**: **public** functions of the **base class** must be used instead.

A **derived class** can *redefine* (define a different function with the same name) any of the member functions of the **base class**.

The function of the **base class** is accessible in the **derived class** by means of the operator (::). The redefined function is directly usable in the **derived class**.

3. Example: class "employee"

Definition of the function `calculate_payment()` in each of the classes

```
float subcontracted::calculate_payment() {  
    return cost_rate * hours;  
};  
float salesman::calculate_payment() {  
    return (subcontracted::calculate_payment() + comission * sales);  
};  
float manager::calculate_payment() {  
    return salary;  
};
```

It is **NOT** correct to use the definitions:

```
float salesman : calculate_payment() {  
    return (cost_rate * hours + comision * sales);  
};  
float salesman : calculate_payment() {  
    return (calculate_payment() + comision * sales);  
};
```

Private

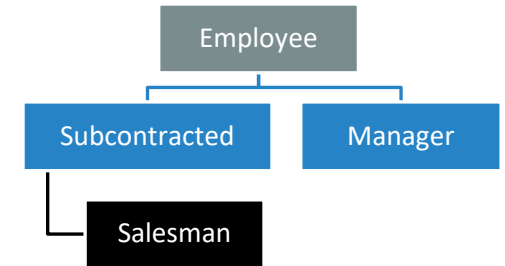
Ambiguous

because `salesman::calculate_payment()` does not have access to the **private** variables of `subcontracted`. The function `calculate_payment()` of `salesman` should not call itself, but the function of `subcontracted`.

3. Example: class "employee"

```
class employee
{
private:
    char *name;
public:
    employee();
    employee(char* n);
    char* return_name();
    float calculate_payment() { return 0.0; }
};
```

```
class manager : public employee
{
private:
    float salary;
public:
    manager();
    manager(char* n, float s);
    void Set_salary(float s);
    float calculate_payment();
};
```



```
class subcontracted : public employee
{
private: protected:
    float cost_rate;
    float hours;
public:
    subcontracted();
    subcontracted(char* n, float c, float h);
    void Set_cost_rate(float c);
    void Set_hours(float h);
    float calculate_payment();
};
```

```
class salesman : public subcontracted
{
private:
    float commision;
    float sales;
public:
    salesman();
    salesman(char* n, float cr, float h, float c, float s);
    void Set_commission(float c);
    void Set_sales(float s);
    float calculate_payment();
};
```

3. Example: class "employee"

Definition of the function **calculate_payment()** in each of the classes

```
float subcontracted::calculate_payment() {  
    return cost_rate * hours;  
};  
float salesman::calculate_payment() {  
    return (cost_rate * hours + commision * sales);  
};  
float manager::calculate_payment() {  
    return salary;  
};
```

In this case **cost_rate** and **hours** have been declared as **protected** → it is possible to access to them from a derived class

4. Derived class constructors

An object of a **derived class** contains all the members of the **base class**. The **constructor** of the **derived class** must call the constructor of the **base class**.

When defining a constructor for a **derived class**, a **base initializer** (call to the **constructor** of the **base class**) must be specified.

The **base initializer** is specified by following the **constructor** arguments with the character (:) and a **constructor of the base class** followed by a list of arguments in parentheses.

Example:

```
// subcontracted constructor
subcontracted(char* n, float c_rate, float h) :employee(n) {
    cost_rate = c_rate;
    hours = h;
}
```

When declaring an object of a **derived class**, the **constructor** of the **base class** is executed first, then the **constructor** of the **derived class**.

The **base initialiser** can be omitted if the **base class** has a **default constructor**.

The **constructor** of a **derived class** must have values for its own variables and for the **constructor** of the **base class**.

4. Derived class constructors

An object of a **derived class** contains all the members of the **base class**. The **constructor** of the **derived class** must call the constructor of the **base class**.

When defining a constructor for a **derived class**, a **base initializer** (call to the **constructor** of the **base class**) must be specified.

The **base initializer** is specified by following the **constructor** arguments with the character (:) and a **constructor of the base class** followed by a list of arguments in parentheses.

Example:

```
// subcontracted constructor
subcontracted(char* n, float c_rate, float h) :employee(n) {
    cost_rate = c_rate;
    hours = h;
}
```

When declaring an object of a **derived class**, the **constructor** of the **base class** is executed first, then the **constructor** of the **derived class**.

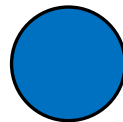
The **base initialiser** can be omitted if the **base class** has a **default constructor**.

The **constructor** of a **derived class** must have values for its own variables and for the **constructor** of the **base class**.

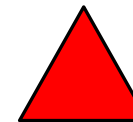
Classes without Inheritance



```
// Circle.h
class Circle{
    double area, perimeter, radius;
public:
    Circle(double a=0, double p=0, double r=0);
    ~Circle();
    void prt();
    void Calculate(void);
};
```



```
// Triangle.h
class Triangle{
    double area, perimeter, base, height;
public:
    Triangle(double a=0, double p=0, double b=0,
double h=0);
    ~Triangle();
    void prt();
};
```



```
// Rectangle.h
class Rectangle{
    double area, perimeter, base, height;
public:
    Rectangle(double a=0, double p=0, double b=0,
double h=0);
    ~Rectangle();
    void prt();
    void Calculate (void);
};
```



```
// Circle.cpp
Circle::Circle(double a, double p, double r) {
    area = a;
    perimeter = p;
    radius = r;
}

Circle::~Circle() {}

void Circle::prt() {
    cout << "Area: " << area;
    cout << "\tPerimeter: " << perimeter;
    cout << "\tRadius: " << radius << endl;
}

void Circle::Calculate (void) {
    area = 3.14 * radius * radius;
    perimeter = 2 * 3.14 * radius;
}
```

```
// Triangle.cpp
Triangle::Triangle(double a, double p, double
b, double h) {
    area = a;
    perimeter = p;
    base = b;
    height = h;
}

Triangle::~Triangle() {}

void Triangle::prt() {
    cout << "Area: " << area;
    cout << "\tPerimeter: " << perimeter;
    cout << "\tBase: " << base;
    cout << "\tHeight: " << height << endl;
}
```

```
// Rectangle.cpp
Rectangle::Rectangle(double a, double p, double
b, double h) {
    area = a;
    perimeter = p;
    base = b;
    height = h;
}

Rectangle::~Rectangle() {}

void Rectangle::prt() {
    cout << "Area: " << area;
    cout << "\tPerimeter: " << perimeter;
    cout << "\tBase: " << base;
    cout << "\tHeight: " << height << endl;
}

void Rectangle::Calculate (void) {
    area = base * height;
    perimeter = 2 * (base + height);
}
```

Classes without Inheritance



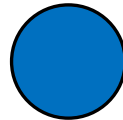
// The same, but all in “.h” file

```
class Circle{
    double area, perimeter, radius;
public:
    Circle(double a=0, double p=0, double r=0) {
        area = a;
        perimeter = p;
        radius = r;
    }

    ~Circle() {}

    void prt() {
        cout << "Area: " << area;
        cout << "\tPerimeter: " << perimeter;
        cout << "\tRadius: " << radius << endl;
    }

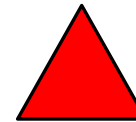
    void Calculate(void) {
        area = 3.14 * radius * radius;
        perimeter = 2 * 3.14 * radius;
    }
};
```



```
class Triangle{
    double area, perimeter, base, height;
public:
    Triangle(double a=0, double p=0, double b=0,
        double h=0) {
        area = a;
        perimeter = p;
        base = b;
        height = h;
    }

    ~Triangle() {}

    void prt() {
        cout << "Area: " << area;
        cout << "\tPerimeter: " << perimeter;
        cout << "\tBase: " << base;
        cout << "\tHeight: " << height << endl;
    }
};
```



```
class Rectangle{
    double area, perimeter, base, height;
public:
    Rectangle(double a=0, double p=0, double b=0,
        double h=0) {
        area = a;
        perimeter = p;
        base = b;
        height = h;
    }

    ~Rectangle() {}

    void prt() {
        cout << "Area: " << area;
        cout << "\tPerimeter: " << perimeter;
        cout << "\tBase: " << base;
        cout << "\tHeight: " << height << endl;
    }

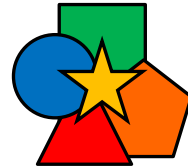
    void Calculate (void) {
        area = base * height;
        perimeter = 2 * (base + height);
    }
};
```



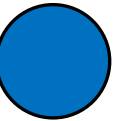
Review: Using Inheritance (1)



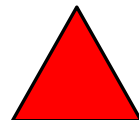
```
class Shape{
protected:
    double area;
    double perimeter;
public:
    Shape(double a=0, double p=0) { area = a; perimeter = p; }
    ~Shape(){}
    void prt() {
        cout << "Area: " << area;
        cout << "\tPerimeter: " << perimeter;
    }
    void Calculate(){ cout << "Not defined" << endl;}
};
```



```
class Circle : public Shape{
    double radius;
public:
    Circle(double a=0, double p=0, double b=0) :Shape(a,p) { radius = r; }
    ~Circle() {}
    void prt() {
        Shape::prt();
        cout << "\tRadius: " << radius << endl;
    }
    void Calculate(void) {
        area = 3.14 * radius * radius;
        perimeter= 2 * 3.14 * radius;
    }
};
```



```
class Triangle : public Shape {
    double base, height;
public:
    Triangle(double a=0, double p=0, double b=0, double h=0) {
        base = b;
        height = h;
    }
    ~Triangle(){}
    void prt() {
        Shape::prt();
        cout << "\tBase: " << base << "\tHeight: " << height << endl;
    }
};
```



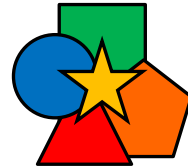
```
class Rectangle : public Shape{
    double base, height;
public:
    Rectangle(double a=0, double p=0, double b=0, double h=0):Shape(a, p){
        base = b;
        height = h;
    }
    ~Rectangle() {}
    void prt() {
        Shape::prt();
        cout << "\tBase: " << base << "\tHeight: " << height << endl;
    }
    void Calculate (void) {
        area = base * height;
        perimeter = 2 * (base + height);
    }
};
```



Review: Using Inheritance (2)



```
class Shape{
protected:
    double area;
    double perimeter;
public:
    Shape(double a=0, double p=0) { area = a; perimeter = p; }
    ~Shape(){}
    void prt() {
        cout << "Area: " << area;
        cout << "\tPerimeter: " << perimeter;
    }
    void Calculate(){ cout << "Not defined" << endl;}
};
```



```
class Triangle : public Shape {
    double base, height;
public:
    Triangle(double a=0, double p=0, double b=0, double h=0) {
        base = b;
        height = h;
    }
    ~Triangle(){}
    void prt() {
        Shape::prt();
        cout << "\tBase: " << base << "\tHeight: " << height << endl;
    }
};
```

Not defined
in Triangle

```
#include "Shape.h"
```

```
void main()
```

```
{
```

```
    Shape s;
```

```
    Circle c;
```

```
    Rectangle r;
```

```
    Triangle t;
```

```
    s.Calculate();
```

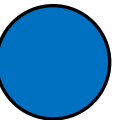
```
    c.Calculate();
```

```
    r.Calculate();
```

```
    t.Calculate();
```

```
}
```

```
class Circle : public Shape{
    double radius;
public:
    Circle(double a=0, double p=0, double b=0) :Shape(a,p) { radius = r; }
    ~Circle() {}
    void prt() {
        Shape::prt();
        cout << "\tRadius: " << radius << endl;
    }
    void Calculate(void) {
        area = 3.14 * radius * radius;
        perimeter= 2 * 3.14 * radius;
    }
};
```



```
class Rectangle : public Shape{
    double base, height;
public:
    Rectangle(double a=0, double p=0, double b=0, double h=0):Shape(a, p){
        base = b;
        height = h;
    }
    ~Rectangle() {}
    void prt() {
        Shape::prt();
        cout << "\tBase: " << base << "\tHeight: " << height << endl;
    }
    void Calculate (void) {
        area = base * height;
        perimeter = 2 * (base + height);
    }
};
```



5. Conversions between objects of base classes and derived classes



It is possible to make **conversions** or **assignments** between an object of a **derived class** to an object of the **base class** (it is possible to go from the most particular to the most general, although information may be lost). It is possible to do:

```
an_employee = a_subcontracted;  
a_subcontracted = a_salesman;
```

```
employee e1((char*)"Paul"), e2;  
subcontracted s1((char*)"Steve", 15, 4), s2;  
e2 = s1;  
s2 = e1; //This one is NOT possible
```

Conversions in the opposite direction (from the most general to the most particular) **are not possible**, as values are not available for all member variables of the derived class and there would be variables that would remain uninitialised).

Similarly, a **pointer to a derived class** can be converted into a **pointer to the base class**.

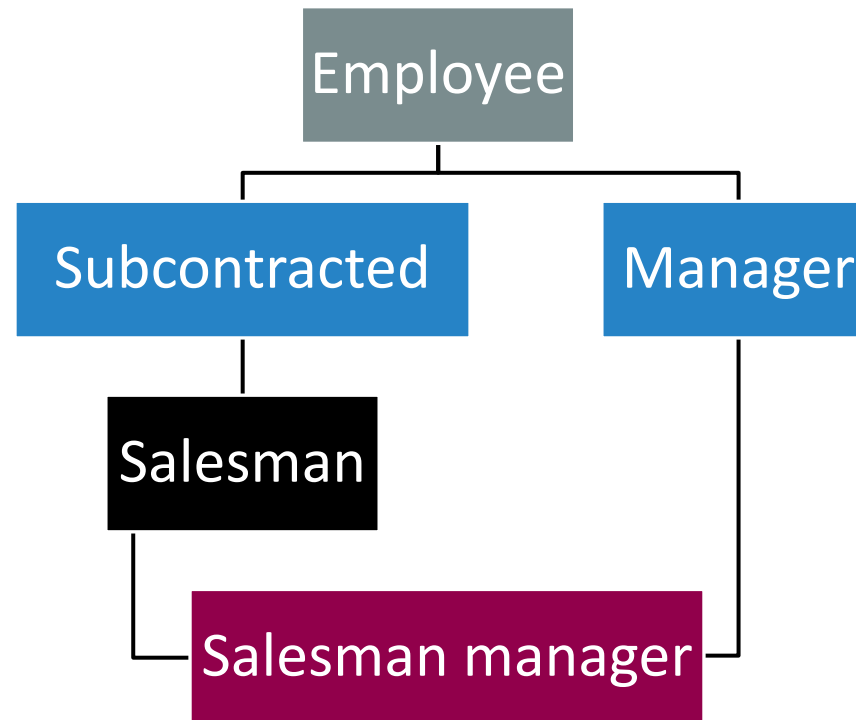
```
employee *e1, e2;  
subcontracted *s1, s2;  
e2 = s1;  
s2 = e1; //This one is NOT possible
```

A pointer to the base class can store the address of an object of any of the classes derived from that base class.

An object of a derived class can be referenced with a pointer to the base class.

When an object is referred to by means of a pointer, the **type of pointer** determines the **member function** that applies to that object, if such a function is defined in the **base** and **derived classes**.

6. Example: class "employee"





6. Multiple inheritance. Virtual base classes

A **derived class** can inherit variables and member functions from several **base classes**.

For example, the class **sales_manager** can inherit the members of the classes **sales_manager** and **salesman**. It would be defined in the form:

```
class sales_manager : public manager, public salesman {  
    // functions and variables definition  
};
```

A **base class** cannot appear directly more than once in the definition of the **derived class**. However, indirectly a class can be a **base class** more than once in a **derived class**.

For example, the class **employee** is a **base class** in duplicate of the class **sales_manager**. The class **sales_manager** has duplicate members of the base class **employee**. The operator "::" can be used.

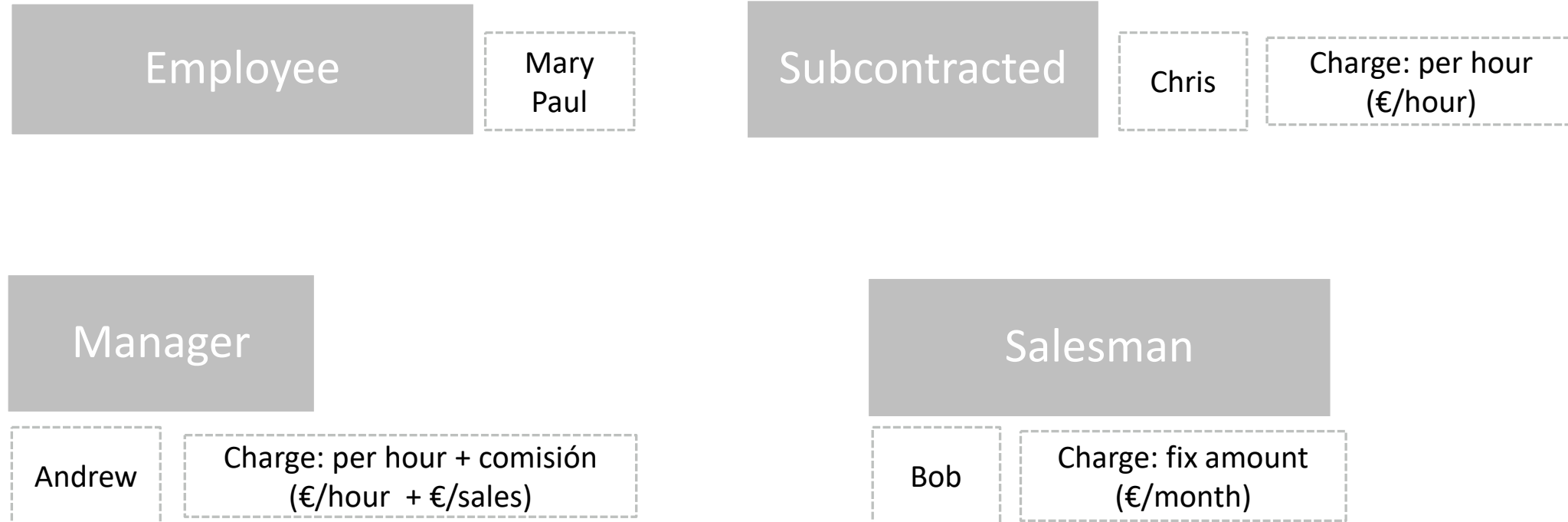
When calling the function **return_name()** it will give an ambiguity error, as it does not know which of the two to use (for manager or for salesman).

To avoid this, the **employee** base class must be declared as a **virtual base class**. In the definitions of the **derived classes**, it is necessary to do the following:

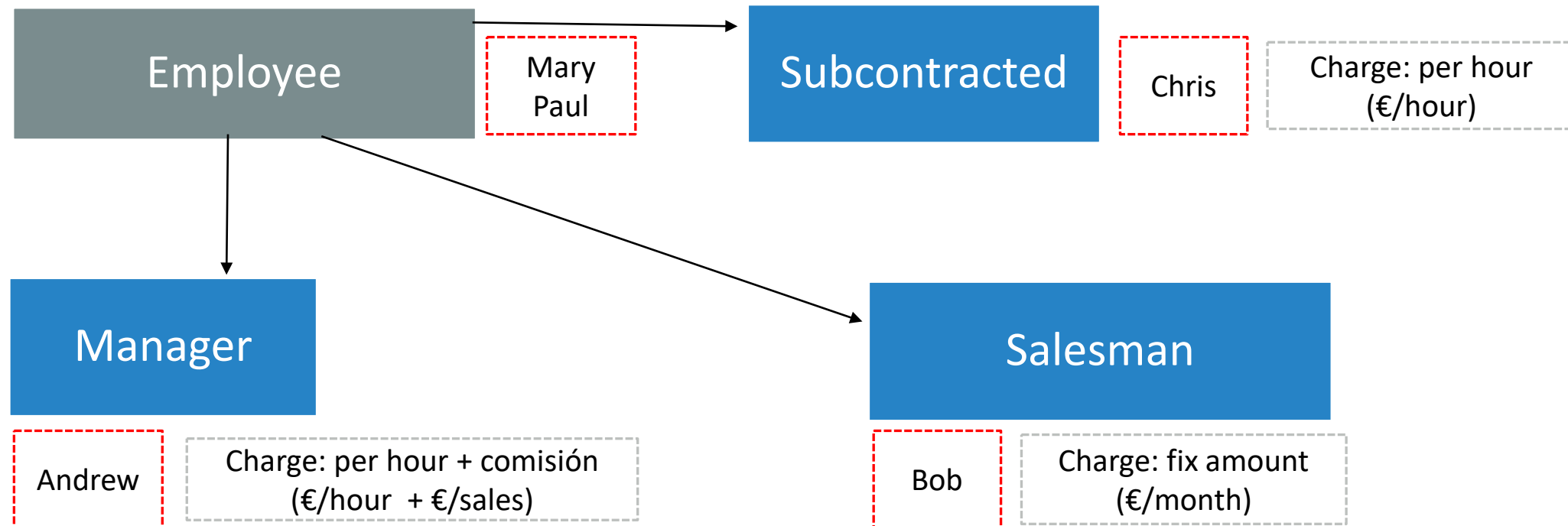
```
class subcontracted : public virtual employee { ... };  
class manager : public virtual employee { ... };
```

The constructor of the **virtual** base class (employee) is always called before the constructors of the **non-virtual** base classes (employee or subcontracted), therefore the constructor of **employee** must be called first: **salesman(...,...,...), employee(...): subcontracted(...){ ; }**

3. Example: class "employee"



3. Example: class "employee"



3. Example: class "employee"

